U-vote: Platform Report

Final Year Project Stage 1

Contributors:

- Luke Doyle (D00255656)
- Hafsa Moin (D00256764)

Date:

February 2026

- Final Year Project Stage 1
  - Contributors:
  - Date:
- 1. Source Control Strategy
  - 1.1 Repository Platform
  - 1.2 Branching Strategy
  - 1.3 Commit and PR Standards
  - 1.4 Repository Structure
  - 1.5 .gitignore Policy
  - 1.6 Day-to-Day Development Workflow
- 2. Environments - Servers, Storage, Network & Security
  - 2.1 Environment Overview
  - 2.2 Cluster Topology
  - 2.3 Storage
  - 2.4 Network Architecture
    - Network Policy Structure
  - 2.5 Security Configuration
    - Kubernetes Secrets
    - TLS and HTTPS
  - 2.6 Resource Requirements
- 3. Pipeline Design
  - 3.1 Pipeline Overview
  - 3.2 Infrastructure Provisioning Script (setup_k8s_platform.py)
  - 3.3 Application Deployment Script (deploy_platform.py)
  - 3.4 Planned CI/CD Pipeline (Stage 2)
- 4. Testing Strategy & Automation
  - 4.1 Testing Philosophy
  - 4.2 Database Test Suite (tests/test_db.py)
    - Test Categories and Coverage
    - Test Execution Options
  - 4.3 Platform Health Tests (deploy_platform.py - Phases 7 & 8)
    - TCP Connectivity Tests (Phase 7)
    - Health Endpoint Tests (Phase 8)
  - 4.4 Manual Testing Evidence
  - 4.5 API Test Suite (tests/test_api.py)
    - Test Suite Structure

Authentication and Fixture Strategy
Test Execution
4.6 Known Test Gaps & Stage 2 Plans
5. Release & Deployment Processes
5.1 Deployment Model
5.2 Deployment Execution
5.3 Docker Image Build Process
5.4 Kubernetes Deployment Configuration
5.5 Release Versioning
5.6 Rollback Procedure
6. Migration Strategy
6.1 Schema Migration Approach
6.2 Stage 2 ,Formal Migration Tooling
6.3 Database Migration for Production
6.4 Data Integrity During Migration
6.5 Cloud Migration Strategy (Stage 2)
Option A - DkIT XOA
Option B - AWS
Option C - Azure
Component Mapping by Target
6.6 Environment Promotion
7. Repo Structure & Container Registries
7.1 Repository Architecture
7.2 Container Registry (Current)
7.3 Container Registry (Stage 2 Plan)
7.4 Docker Image Design

---

1. Source Control Strategy

1.1 Repository Platform

UVote uses GitHub as its version control platform. The repository is hosted at GitHub - D00256764/u-vote and is organised as a mono-repository containing all platform code, application code, Kubernetes manifests, Python automation scripts, and documentation references.

The mono-repo approach was chosen because all six microservices share a common asyncpg database module (shared/database.py), a common security module (shared/security.py), and identical Dockerfile patterns. Keeping all services in a single repository simplifies shared-module management, avoids cross-repository dependency coordination, and ensures that all Kubernetes manifests are always in sync with the service code they deploy.

1.2 Branching Strategy

The repository follows a simplified trunk-based branching model suitable for a two-person team:

- **main** - the stable, deployable branch. Direct pushes to main are restricted. All changes must arrive via pull request.

- **develop** - the integration branch where completed features are merged before promotion to main.
- **feature/*** - short-lived branches created from develop for individual features or fixes. Named with a descriptive slug, e.g., `feature/token-generation`, `feature/calico-network-policies`.
- **hotfix/*** - branches cut from main for urgent production fixes, merged back to both main and develop.

1.3 Commit and PR Standards

Commit messages follow the Conventional Commits format: `type(scope): description`. Common types: `feat`, `fix`, `docs`, `test`, `chore`, `refactor`.

Pull requests require at least one reviewer approval before merge. The reviewer verifies functional correctness, code style, and documentation completeness.

Each pull request is linked to a GitHub Issue describing the requirement or bug it addresses.

The branch is deleted after merge to keep the repository clean.

1.4 Repository Structure

The repository is organised as follows, using the Linux `tree` command:

```
/u-vote$ tree
├── auth-service
│   ├── app.py
│   ├── Dockerfile
│   └── requirements.txt
├── database
│   └── init.sql
├── docker-compose.yml
├── election-service
│   ├── app.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── static
│   │   └── css
│   │       └── style.css
│   └── templates
│       ├── base.html
│       ├── create_election.html
│       ├── dashboard.html
│       └── election_detail.html
├── frontend-service
│   ├── app.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── static
│   │   └── css
│   │       └── style.css
│   └── templates
│       ├── base.html
│       ├── index.html
│       ├── login.html
│       ├── manage_voters.html
│       └── register.html
```

```
├── plat_scripts
│   ├── dashboard.py
│   ├── deploy_platform.py
│   ├── port_forward.py
│   ├── __pycache__
│   ├── requirements.txt
│   └── setup_k8s_platform.py
├── README.md
├── results-service
│   ├── app.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── static
│   │   └── css
│   │       └── style.css
│   └── templates
│       ├── base.html
│       └── results.html
├── sample-voters.csv
├── shared
│   ├── database.py
│   ├── email_util.py
│   ├── schemas.py
│   └── security.py
├── tests
│   ├── logs
│   ├── __pycache__
│   ├── test_api.py
│   └── test_db.py
├── uvote-platform
│   ├── k8s
│   │   ├── database
│   │   │   ├── db-deployment.yaml
│   │   │   ├── db-pvc.yaml
│   │   │   ├── db-secret.yaml
│   │   │   └── schema.sql
│   │   ├── ingress
│   │   │   └── uvote-ingress.yaml
│   │   ├── namespaces
│   │   │   └── namespaces.yaml
│   │   ├── network-policies
│   │   │   ├── 00-default-deny.yaml
│   │   │   ├── 01-allow-dns.yaml
│   │   │   ├── 02-allow-to-database.yaml
│   │   │   ├── 03-allow-from-ingress.yaml
│   │   │   ├── 04-allow-audit.yaml
│   │   │   └── test-pods.yaml
│   │   ├── secrets
│   │   └── services
│   │       ├── admin-deployment.yaml
│   │       ├── auth-deployment.yaml
│   │       ├── election-deployment.yaml
│   │       ├── frontend-deployment.yaml
│   │       ├── results-deployment.yaml
│   │       └── voting-deployment.yaml
│   └── kind-config.yaml
├── voter-service
│   ├── app.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── static
│   │   └── css
│   │       └── style.css
│   └── templates
│       ├── base.html
│       └── manage_voters.html
└── voting-service
    ├── app.py
    ├── Dockerfile
    ├── requirements.txt
    ├── static
    │   └── css
```

```
    │         └── style.css
    └── templates
        ├── base.html
        ├── verify_identity.html
        ├── vote_error.html
        ├── vote.html
        ├── vote_success.html
        └── vote_verified.html
```

1.5 .gitignore Policy

The following are excluded from version control: Python virtual environments (`.venv/` , `__pycache__/`), environment variable files (`.env`), compiled Python files (`.pyc`), Docker build caches, and IDE configuration directories (`.vscode/` , `.idea/`). Kubernetes Secrets are never committed - they are created at deploy time by the `deploy_platform.py` script.

1.6 Day-to-Day Development Workflow

All development work follows a consistent process from issue creation through to promotion on main. The workflow is as follows:

Step	Action	Detail
1	Create GitHub Issue	A GitHub Issue is opened describing the feature, bug, or infrastructure change. The issue includes acceptance criteria, any relevant ADR references, and is assigned to a team member.
2	Branch from develop	The assigned developer creates a `feature/*` branch from the current HEAD of develop, using a descriptive slug that matches the issue (e.g., `feature/blind-ballot-token` or `fix/vote-trigger-exception`).
3	Develop and commit	Work is committed incrementally using Conventional Commits (e.g., `feat(voting): implement blind ballot token generation`). Commits are kept atomic, one logical change per commit.
4	Open pull request	When the feature is ready, a pull request is opened against develop. The PR description links the GitHub Issue, summarises the changes,

		and lists any manual tests performed. If Kubernetes manifests or schema.sql were changed, this is explicitly noted.
5	Peer review	The other team member reviews the PR, checking functional correctness, code style (PEP 8), security (no hardcoded secrets, no direct postgres calls from the frontend), and documentation completeness. Comments are resolved before approval.
6	Merge to develop	After approval, the PR is squash-merged into develop and the feature branch is deleted. The integration branch now contains the completed feature alongside any other in-progress work.
7	Integration validation	With the feature on develop, the team runs `python3 tests/test_db.py --quick` and `python3 plat_scripts/deploy_platform.py --verbose` to confirm the change deploys cleanly and all health checks pass.
8	Promote to main	Once develop is stable and the sprint milestone is met, a pull request is opened from develop to main. This requires a second review pass. On approval, the PR is merged - this constitutes a formal release. No direct commits to main are permitted at any stage.

Hotfix branches follow the same review and merge process but are cut from main rather than develop, and are merged back into both main and develop on completion.

---

2. Environments - Servers, Storage, Network & Security

2.1 Environment Overview

UVote maintains three logical Kubernetes namespaces that map to development, testing, and production environments within the same Kind cluster. This allows environment separation, namespace-scoped Secrets, and independent deployment without requiring separate physical infrastructure during Stage 1.

Namespace	Purpose	Services Deployed	Access
uvote-dev	Day-to-day development and feature testing. Hot-reloaded containers where applicable.	All 6 services + PostgreSQL	Team members only
uvote-test	Integration testing and pre-release validation. Matches production configuration.	All 6 services + PostgreSQL	Team members + CI runner
uvote-prod	Stable production-grade deployment. Identical manifests to test with production Secrets.	All 6 services + PostgreSQL	Restricted - admin approval required

---

2.2 Cluster Topology

Component	Configuration	Notes
Distribution	Kind (Kubernetes IN Docker)	Local development cluster - runs inside Docker on the development machine
Control Plane	1 node	Manages scheduling, API server, etcd
Worker Nodes	2 nodes	Service pods distributed across both workers for HA
CNI Plugin	Calico v3.26.1	Installed with default CNI disabled; iptables data plane; VXLAN encapsulation
Pod CIDR	192.168.0.0/16	Calico default CIDR configured in kind-config.yaml

Ingress Controller	Nginx (Helm chart)	Deployed to the ingress-nginx namespace; handles TLS termination and routing

---

2.3 Storage

Resource	Type	Size	Access Mode	Purpose
PostgreSQL PVC	PersistentVolumeClaim	5Gi	ReadWriteOnce	Primary database storage - survives pod restarts
Seed data	ConfigMap / schema.sql	N/A	N/A	Applied once at cluster setup via kubectl exec
Docker image layers	Kind image cache	Variable	N/A	Built locally and loaded into Kind with `kind load docker-image`

2.4 Network Architecture

The network enforces a zero-trust model. The default policy is deny-all for all pod-to-pod traffic within each namespace. Twelve explicit Network Policy resources define the permitted communication paths. All external traffic enters through the Nginx Ingress Controller.

Network Policy Structure

Policy	Allows	Denies
default-deny-all	Nothing (baseline deny)	All pod-to-pod traffic not explicitly permitted
allow-frontend-to-gateway	Frontend pod to reach Nginx	Direct backend access from frontend
allow-backend-to-postgres	Auth, Voting, Election, Results, Audit, Admin to	Frontend to PostgreSQL (explicitly blocked)

	reach PostgreSQL on port 5432	
allow-admin-to-email	Admin Service to reach Email Service on port 8007	All other services to Email Service
allow-ingress-to-services	Nginx Ingress to reach all services on their designated ports	Direct pod-to-pod requests bypassing Ingress

---

2.5 Security Configuration

Kubernetes Secrets

Secret Name	Contents	Used By
db-credentials	POSTGRES_USER, POSTGRES_PASSWORD, DB_HOST, DB_NAME	All six microservices
jwt-secret	JWT_SECRET_KEY	Auth Service, API Gateway JWT validation
flask-secret	SECRET_KEY	Frontend Service session management
smtp-credentials	SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD	Email Service only

Secrets are created at deploy time by `deploy_platform.py` using `kubectl create secret`. They are never written to YAML files or committed to version control. Each pod's spec references the Secret via `envFrom` or `env[].valueFrom.secretKeyRef`.

TLS and HTTPS

Stage 1 runs without TLS as the cluster is local and not publicly reachable. The Nginx Ingress is configured to accept HTTP on port 80. The architecture is designed for TLS to be added in Stage 2 using cert-manager with a Let's Encrypt issuer. The Nginx Ingress configuration will be updated with a `tls:` block referencing a TLS Secret, and cert-manager will automate certificate provisioning and renewal.

2.6 Resource Requirements

The following resource profile is the result of empirical measurement during Stage 1 development:

Component	CPU Request	CPU Limit	Memory Request	Memory Limit	Replicas
FastAPI services (each)	100m	500m	128Mi	256Mi	2
Frontend Service	100m	500m	128Mi	256Mi	2
PostgreSQL	200m	1000m	256Mi	512Mi	1
Nginx Ingress	100m	500m	128Mi	256Mi	1
Calico (system pods)	~100m total	~300m total	~200Mi total	~400Mi total	3–4

Total cluster resource estimate: approximately 4–6 GB RAM and 2–4 CPU cores. This is within the constraint of the development machines (16 GB RAM, 8-core CPU).

---

3. Pipeline Design

3.1 Pipeline Overview

Stage 1 uses a manual pipeline implemented as three Python scripts that are executed in a defined order. The pipeline is designed to be run by a developer on a local machine with Docker and kubectl installed. Automated CI/CD via GitHub Actions is a planned Stage 2 enhancement.

Stage	Script	Purpose
Infrastructure	plat_scripts/setup_k8s_platform.py	Provision Kind cluster, install Calico, deploy PostgreSQL, apply schema, install NetworkPolicies
Database Validation	tests/test_db.py	Validate schema correctness, seed data,

		trigger behaviour, security controls, and network connectivity
Application Deployment	plat_scripts/deploy_platform.py	Build Docker images, load into Kind, create Secrets, deploy services, run health checks

---

3.2 Infrastructure Provisioning Script (setup_k8s_platform.py)

This script automates the complete cluster setup from a clean machine state. It must be run before the other two scripts. Each step is implemented as a discrete Python function and logged with pass/fail status.

Step	Function	What It Does
0	check_prerequisites	Verifies docker, kubectl, kind, and helm are on the system PATH
1	create_kind_cluster	Creates the Kind cluster 'uvote' from kind-config.yaml (1 control-plane, 2 workers, Calico CIDR, default CNI disabled)
2	install_calico	Applies the Calico tigera-operator.yaml and custom-resources.yaml; waits up to 300 seconds for all Calico pods to reach Ready state
3	apply_namespaces	Applies namespaces.yaml to create uvote-dev, uvote-test, and uvote-prod namespaces
4	deploy_database	Applies the db-secret, db-pvc, and db-deployment Kubernetes manifests to the target namespace
5	apply_database_schema	Waits for the PostgreSQL pod to be Running, then pipes schema.sql into the pod via kubectl exec and psql
6	apply_network_policies	Applies all *.yaml files in the network-policies/ directory in sorted alphabetical order

7	install_ingress_controller	Installs the ingress-nginx Helm chart from the stable repository
8	verify_setup	Reports the state of nodes, Calico pods, the PostgreSQL pod, and all namespaces - informational only

---

3.3 Application Deployment Script (deploy_platform.py)

This script builds, packages, and deploys all six UVote microservices to the Kind cluster. It runs in 9 sequential phases, each of which must succeed before the next begins.

Phase	Action	Success Criterion
1	Pre-flight checks	Kind cluster accessible; Docker daemon running; target namespace exists; PostgreSQL pod Ready
2	Build Docker images	All 6 service images built successfully within 600-second timeout per image
3	Load images into Kind	All images loaded via `kind load docker-image` without error
4	Create/update Secrets	db-credentials, jwt-secret, flask-secret created or updated; PostgreSQL password synchronised
5	Apply service manifests	All Deployment and Service YAML files in k8s/services/ applied successfully
6	Poll pod readiness	All service pods reach Running state with containers ready within --timeout (default 300s)
7	Test TCP connectivity	Each backend pod can reach postgresql:5432; frontend pod connection is blocked by NetworkPolicy
8	Test health endpoints	Port-forward tunnel opened per service; /health returns HTTP 200 for all services

9	Print summary	Pass/fail counts printed; script exits 0 on all-pass, non-zero on any failure

---

3.4 Planned CI/CD Pipeline (Stage 2)

The following GitHub Actions pipeline is planned for Stage 2 to automate the full build-test-deploy cycle on every pull request to develop and every merge to main:

Stage	Trigger	Actions
Lint & Static Analysis	Pull request to develop or main	Run flake8 (Python style), bandit (security linting), and Pydantic v2 compatibility checks on all service code
Unit Tests	Pull request to develop or main	Run pytest against each service in isolation using a test database (PostgreSQL running in a GitHub Actions service container)
Build Images	Pull request to develop or main	Build all 6 Docker images; tag with commit SHA; push to GitHub Container Registry (GHCR)
Integration Tests	Merge to develop	Provision a Kind cluster in the CI runner, deploy services from GHCR images, run tests/test_db.py and deploy_platform.py health checks
Deploy to Production	Merge to main (manual approval)	Apply Kubernetes manifests using the new image tags; verify health probes before declaring success

---

4. Testing Strategy & Automation

4.1 Testing Philosophy

The UVote testing strategy is structured across four layers that validate successively higher abstraction levels: infrastructure correctness, database security and integrity, application service health, and API contract correctness. All test scripts are written in Python for cross-platform compatibility and to match the team's primary language.

The principle of test ordering is enforced by design: `test_db.py` depends on a live PostgreSQL pod (deployed by `setup_k8s_platform.py`), `deploy_platform.py` depends on the cluster, database, and network policies all being in place, and `test_api.py` depends on all services being deployed and healthy. Running tests out of order produces infrastructure failures that present as test failures.

4.2 Database Test Suite (tests/test_db.py)

The `test_db.py` script is the most comprehensive validation layer. It connects to the PostgreSQL pod via `kubectl exec` and executes SQL directly, ensuring that all database-level security controls are validated at the database layer rather than through the application.

Test Categories and Coverage

Category	Tests	Coverage
Infrastructure Health	1, 2	Pod running state; PostgreSQL version connectivity
Schema Correctness	3, 10	All 7 tables present; foreign key constraints in information_schema
Seed Data	4	Admins, elections, and candidates tables populated with development seed data
Security Controls - Triggers	5	UPDATE on votes raises exception; DELETE on votes raises exception; INSERT on votes succeeds
Security Controls - Hash Chain	6	vote_hash is non-NULL in at least one of the last 5 votes after a test INSERT
Security Controls - RBAC	7	auth_service user can SELECT admins and is denied on votes; results_service user can SELECT votes and is denied on INSERT

Query Functionality	8	Multi-table JOIN across elections, candidates, and votes returns rows with a candidate column
Performance Indexes	9	At least one critical index exists (idx_votes_election, idx_votes_candidate, or idx_tokens_token)
Network Connectivity	11	Temporary pod with auth-service label can reach postgresql:5432 via ClusterIP
Load Performance	12	Bulk INSERT of configurable N votes; distribution query returns results (disabled by default)

Test Execution Options

```
python3 tests/test_db.py                  # Full suite including pod-to-pod network path test (Test 11)
python3 tests/test_db.py --quick          # Skips Test 11 (pod network test) for faster iteration
python3 tests/test_db.py --load-test N   # Enables Test 12 with N votes inserted (use only on dedicated VM)

```

4.3 Platform Health Tests (deploy_platform.py - Phases 7 & 8)

TCP Connectivity Tests (Phase 7)

For each backend service, the script launches a temporary test pod and attempts a TCP connection to the ClusterIP of the PostgreSQL service on port 5432. The expected results are:

- **auth-service pod:** TCP CONNECT succeeds (permitted by NetworkPolicy).
- **voting-service pod:** TCP CONNECT succeeds.
- **election-service pod:** TCP CONNECT succeeds.
- **results-service pod:** TCP CONNECT succeeds.
- **audit-service pod:** TCP CONNECT succeeds.
- **admin-service pod:** TCP CONNECT succeeds.
- **frontend-service pod:** TCP CONNECT refused/timeout (blocked by default-deny NetworkPolicy - expected failure confirms security policy is working).

Health Endpoint Tests (Phase 8)

For each service, the script opens a `kubectl port-forward` tunnel and sends an HTTP GET to `/health`. The expected response is HTTP 200 with a JSON body containing `status:`

`healthy`. This confirms that the FastAPI application has started, the asyncpg pool has been initialised, and the service is accepting requests.

4.4 Manual Testing Evidence

Feature	Test Method	Result
Admin registration	POST /api/auth/register via curl and frontend form	Account created; bcrypt hash stored; duplicate email rejected
Admin login / JWT	POST /api/auth/login; inspect JWT payload	JWT returned with correct sub, email, role, iat, exp claims
Election creation	POST /api/elections via admin dashboard	Election created in draft status; listed on dashboard
Candidate management	POST, PUT, DELETE /api/admin/candidates	CRUD operations confirmed; minimum 2 candidate activation rule enforced
CSV voter import	POST /api/admin/voters/bulk with test CSV	All voters imported; duplicate email per election rejected by UNIQUE constraint
Token generation	POST /api/admin/elections/{id}/send-voting-links	43-character tokens generated; stored with 7-day expiry; emails dispatched
Token validation	GET /api/voting/validate-token?token={valid}	Returns election metadata; expired token returns error; used token returns has_voted: true
Vote submission	POST /api/voting/cast-vote	Vote recorded; token marked used; hash

		chain computed; audit log entry written
Double vote prevention	POST /api/voting/cast-vote (second attempt)	Returns 'already voted' - voting_tokens UNIQUE constraint prevents second token issuance
Vote anonymity	Direct psql query on votes table	No voter_id column present; no JOIN possible from votes to voters
Vote immutability	UPDATE votes SET candidate_id=1 WHERE vote_id=1	PostgreSQL raises 'Votes cannot be modified or deleted' exception
Results calculation	GET /api/results/{id} after election closed	Correct tallies, percentages, and winner returned
Audit log immutability	UPDATE audit_logs SET details='' WHERE log_id=1	PostgreSQL raises 'Audit log records cannot be modified or deleted' exception

---

4.5 API Test Suite (tests/test_api.py)

The `test_api.py` suite provides automated API-level validation using FastAPI's TestClient backed by an in-process test database. It occupies the layer between the database tests (`test_db.py`) and the deployment smoke tests (`deploy_platform.py`), confirming that each service's HTTP interface behaves correctly before a deployment is committed to develop or main.

The suite is structured around the six services and targets the endpoints that represent core business logic. Each test sends an HTTP request through the FastAPI application stack, including middleware, authentication guards, and database operations, and asserts on the response status code, response body structure, and relevant side effects (such as database state changes or header values).

Test Suite Structure

Service / Module	Endpoints Tested	What Each Test Validates
Auth Service	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me	Registration stores a bcrypt hash and returns 201; login returns a valid JWT on correct credentials and 401 on incorrect credentials; /me returns the decoded token payload for a bearer-authenticated request
Election Service	POST /api/elections, GET /api/elections, PATCH /api/elections/{id}/status	Election creation requires a valid admin JWT and returns 201 with the new election ID; listing returns all elections for the authenticated admin; status transition (draft → active → closed) enforces ordering rules
Admin Service	POST /api/admin/candidates, POST /api/admin/voters/bulk, POST /api/admin/elections/{id}/send-voting-links	Candidate creation enforces a minimum of 2 candidates before activation; CSV voter import rejects malformed rows and duplicate emails within the same election; token generation returns 202 and dispatches one token per voter
Voting Service	GET /api/voting/validate-token, POST /api/voting/cast-vote	Valid token returns election metadata with has_voted: false;

		expired token returns 401; casting a vote returns 200, marks the token as used, and a second cast attempt with the same token returns 409
Results Service	GET /api/results/{id}	Results endpoint returns 403 for an active (open) election and 200 with correct vote tallies and a winner field after the election is closed
Audit Service	GET /api/audit/{election_id}	Returns a chronologically ordered log of audit events for the election; verifies that each log entry contains a SHA-256 hash and that the hash chain is unbroken (each entry's previous_hash matches the prior entry's vote_hash)

Authentication and Fixture Strategy

The test suite uses a pytest fixture hierarchy. A session-scoped fixture provisions a clean PostgreSQL test database and runs `schema.sql` to initialise it. A function-scoped fixture seeds a known admin account and one election per test, rolling back after each test to prevent state leakage between cases. JWT tokens for authenticated requests are generated by calling the auth service's `/login` endpoint rather than being constructed directly, ensuring that the full token pipeline is exercised.

Test Execution

```
python3 -m pytest tests/test_api.py           # Full API suite
python3 -m pytest tests/test_api.py -v        # Verbose output with per-test pass/fail
python3 -m pytest tests/test_api.py -k auth   # Run only auth service tests
```

```
```

The API suite is a prerequisite for opening a pull request to develop. It is run locally by the developer before requesting review, and is the first test target in the planned Stage 2 GitHub Actions pipeline.

4.6 Known Test Gaps & Stage 2 Plans

Unit tests for individual service functions are not yet implemented. Planned using pytest with FastAPI's TestClient and a test PostgreSQL database in Stage 2.

Test 7 in `test_db.py` covers only two of the six database roles (`auth_service` and `results_service`). The remaining four roles (`voting_service` , `election_service` , `audit_service` , `admin_service`) will be added in Stage 2.

End-to-end browser testing (e.g., Playwright or Selenium) is not implemented. Planned as an optional enhancement if time permits.

Test 12 (load performance) is disabled by default due to its resource requirements. A dedicated VM with at least 16 GB RAM is recommended. Votes inserted are permanent due to the immutability trigger.

---

5. Release & Deployment Processes

5.1 Deployment Model

Stage 1 deployments are performed manually using the `deploy_platform.py` script. The script encapsulates the complete build-deploy-verify cycle: it builds Docker images, loads them into the Kind cluster, manages Kubernetes Secrets, applies service manifests, waits for pod readiness, and verifies health endpoints. The entire cycle typically completes in 8–12 minutes on the development machine.

5.2 Deployment Execution

The complete deployment sequence is:

```
# Step 1: Provision infrastructure (first time only)
python3 plat_scripts/setup_k8s_platform.py

# Step 2: Validate database (skip slow network test for speed)
python3 tests/test_db.py --quick

# Step 3: Build, deploy, and smoke-test all services
python3 plat_scripts/deploy_platform.py --verbose

# Step 4: Full database validation including pod-to-pod network path
python3 tests/test_db.py

```

5.3 Docker Image Build Process

Each of the six services has a Dockerfile based on the `python:3.11-slim` base image. The Dockerfile follows a multi-stage pattern that minimises the image size: it copies the shared/ module, installs dependencies from requirements.txt (pinned versions), copies the service code, and sets the Uvicorn entrypoint.

```
FROM python:3.11-slim
WORKDIR /app
COPY shared/ ./shared/
COPY services/auth-service/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY services/auth-service/ .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]

```

5.4 Kubernetes Deployment Configuration

Each service is defined by a Kubernetes Deployment manifest specifying replicas, resource limits, health probes, and environment variable injection from Secrets. An example configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: uvote-prod
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    spec:
      containers:
      - name: auth-service
        image: uvote/auth-service:latest
        ports:
        - containerPort: 8001
        envFrom:
        - secretRef:
            name: db-credentials
        - secretRef:
            name: jwt-secret
        livenessProbe:
          httpGet:
            path: /health
            port: 8001
          initialDelaySeconds: 10
          periodSeconds: 30
        readinessProbe:
          httpGet:
            path: /health
            port: 8001
          initialDelaySeconds: 5
          periodSeconds: 10
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi
```

```
44
```

5.5 Release Versioning

Stage 1 uses a `latest` image tag strategy, where all images are rebuilt and tagged as `latest` on each deployment. Stage 2 will introduce semantic versioning (`vMAJOR.MINOR.PATCH`) and image tagging with both the semantic version and the git commit SHA. The GitHub Container Registry (GHCR) will store tagged image versions, and Kubernetes manifests will reference specific image tags rather than `latest` to ensure reproducibility.

5.6 Rollback Procedure

Stage 1 rollback is performed by re-running `deploy_platform.py` with the previous version of the service code. The Kubernetes RollingUpdate strategy ensures that the previous pods are terminated only after the new pods are confirmed ready, so a failed deployment can be remediated by reverting the image and re-running the deployment script. Stage 2 will introduce formal `kubectl rollout undo` support.

---

6. Migration Strategy

6.1 Schema Migration Approach

Stage 1 uses a manual schema management approach. The database schema is defined in a single `schema.sql` file containing all `CREATE TABLE`, `CREATE INDEX`, `CREATE TRIGGER`, `CREATE EXTENSION`, and `GRANT` statements. The schema is applied once at cluster setup via the `apply_database_schema` step in `setup_k8s_platform.py`.

During Stage 1 development, schema changes are applied manually by connecting to the PostgreSQL pod and running `ALTER TABLE` or `CREATE` statements via psql. This approach is adequate while the schema is still evolving rapidly and the team is the only consumer of the database.

6.2 Stage 2 ,Formal Migration Tooling

If schema change frequency increases in Stage 2, Alembic (the SQLAlchemy migration tool) will be introduced. Alembic will manage a versioned migration history in an `alembic_version` table, allowing incremental schema changes to be applied in order. Each migration will have an `upgrade()` and `downgrade()` function, enabling safe forward and backward migration.

The introduction of Alembic will require a one-time conversion of `schema.sql` into an initial Alembic revision. Subsequent changes will be written as incremental revisions rather than manual `ALTER TABLE` statements.

6.3 Database Migration for Production

The planned process for applying schema migrations to a production environment is:

1. Schema changes are implemented as a new Alembic revision in a feature branch.
2. The revision is reviewed and approved via pull request before merge to develop.
3. Integration tests run the migration against a test database to confirm correctness.
4. The migration is promoted to main and applied to the production database via a pre-deployment hook in the CI/CD pipeline.
5. The Kubernetes deployment rolls out the new service images after the migration completes successfully.

6.4 Data Integrity During Migration

The vote immutability trigger (`prevent_vote_update` , `prevent_vote_delete`) and the audit log immutability trigger must be preserved across all schema migrations. Any migration that touches the `votes` or `audit_logs` tables must verify that trigger functionality is not disrupted. Alembic migration scripts will include explicit trigger verification steps.

6.5 Cloud Migration Strategy (Stage 2)

The Kubernetes manifests are designed for portability. The same YAML files, with minor modifications to image references and storage class specifications, can be applied to any target infrastructure that supports Kubernetes. Three deployment targets were evaluated for Stage 2: DkIT XOA, AWS, and Azure.

Option A - DkIT XOA

XOA is the DkIT-hosted virtualisation platform, providing virtual machines on institutional infrastructure at no cost to the project. It is the primary candidate for Stage 2 deployment. UVote would run on one or more XOA Virtual Machines, with Kind potentially being replaced by a self-managed Kubernetes cluster deployed directly on the VM instances.

The primary issue with XOA is network access, the platform is not directly reachable from outside the DkIT network without routing through the college VPN (which is not compatible with Linux), which adds problems to external testing and CI/CD integration. This is manageable within the project scope, as the team has institutional network access and the CI runner can be configured to connect

via VPN. The absence of licensing or credit costs makes XOA the most sustainable option for the full project duration.

Option B - AWS

AWS is the strongest commercial option in terms of Kubernetes tooling maturity, documentation quality, and the breadth of managed services available. The planned path would use EKS for Kubernetes, RDS for PostgreSQL, and AWS SES for email delivery.

However, the student AWS credits expire in March 2026, which does not cover the full Stage 2 timeline. Continued use after expiry would require personal billing.

Option C - Azure

Azure student accounts carry significant restrictions that prevent use of several services required by UVote, including features relied upon in standard Azure Kubernetes Service tutorials. This was confirmed during the team's prior experience with Azure in second year. Combined with the quality of Azure's student-tier documentation, Azure is deprioritised relative to the other two options. It is not excluded entirely.

Component Mapping by Target

Component	Current (Stage 1)	XOA (Option A)	AWS (Option B)	Azure (Option C)
Kubernetes	Kind (local Docker)	k3s on XOA VM	EKS	AKS
PostgreSQL	Pod in Kind + PVC	Pod on k3s + PVC	RDS (managed)	Azure Database for PostgreSQL
Image registry	Local Kind cache	GHCR	GHCR or ECR	GHCR or ACR
Secrets	Kubernetes Secrets (manual)	Kubernetes Secrets	AWS Secrets Manager	Azure Key Vault
TLS certificates	None (local only)	cert-manager + Let's Encrypt	cert-manager + Let's Encrypt	cert-manager + Let's Encrypt
SMTP	Gmail SMTP relay	Gmail SMTP relay	AWS SES	SendGrid
External access	kubectl port-forward	DkIT VPN	Public load balancer	Public load balancer

Cost	None	None (institutional)	Credits expire March 2026	Restricted student tier

---

6.6 Environment Promotion

The planned Stage 2 environment promotion process follows a dev → test → prod pipeline. Changes are first deployed to uvote-dev (automatically on merge to develop), then promoted to uvote-test for integration testing (automatically after all tests pass), and finally promoted to uvote-prod (manually approved by the project lead). Each promotion uses the same Kubernetes manifests with environment-specific values supplied via namespace-scoped Secrets.

---

7. Repo Structure & Container Registries

7.1 Repository Architecture

As detailed in Section 1.4, UVote uses a mono-repository structure. The decision to use a mono-repo over a multi-repo (one repository per service) was made for the following reasons:

- Shared modules (`database.py` , `security.py`) are imported directly by all services - no package registry or versioning is required.
- A single pull request can update multiple services and their shared dependencies atomically, preventing version drift.
- The Kubernetes manifests are co-located with the application code, ensuring that infrastructure and application changes are always in sync.
- CI/CD pipeline configuration is maintained in a single `.github/workflows/` directory, simplifying the build matrix.

7.2 Container Registry (Current)

Stage 1 does not use a remote container registry. Docker images are built locally by `deploy_platform.py` and loaded directly into the Kind cluster's internal image store using `kind load docker-image`. This approach avoids the need for registry authentication during development and is appropriate for a local cluster.

7.3 Container Registry (Stage 2 Plan)

Stage 2 will introduce GitHub Container Registry (GHCR) as the image registry. GHCR is integrated with the GitHub repository and does not require a separate account or billing for public repositories. Images will be tagged with the git commit SHA and pushed to GHCR by the CI/CD pipeline after a

successful build. Kubernetes manifests will reference specific GHCR image tags rather than locally loaded images.

Image	Stage 1 Tag	Stage 2 Tag Pattern
uvote/auth-service	latest (local)	ghcr.io/d00256764/uvote/auth-service:v1.2.3-abc1234
uvote/voting-service	latest (local)	ghcr.io/d00256764/uvote/voting-service:v1.2.3-abc1234
uvote/election-service	latest (local)	ghcr.io/d00256764/uvote/election-service:v1.2.3-abc1234
uvote/results-service	latest (local)	ghcr.io/d00256764/uvote/results-service:v1.2.3-abc1234
uvote/audit-service	latest (local)	ghcr.io/d00256764/uvote/audit-service:v1.2.3-abc1234
uvote/admin-service	latest (local)	ghcr.io/d00256764/uvote/admin-service:v1.2.3-abc1234
uvote/frontend-service	latest (local)	ghcr.io/d00256764/uvote/frontend-service:v1.2.3-abc1234

7.4 Docker Image Design

All service images are based on `python:3.11-slim`, which provides a minimal Debian base with Python 3.11 pre-installed. The slim variant is used rather than the full `python:3.11` image to reduce image size by approximately 800 MB. Each image layer is ordered from least-frequently-changed to most-frequently-changed to maximise Docker build cache reuse:

- **Layer 1:** Base image (`python:3.11-slim`)
- **Layer 2:** System dependencies (if any native libraries are required)
- **Layer 3:** Python requirements installation (`requirements.txt` - changes infrequently)
- **Layer 4:** Shared module copy (`shared/` - changes occasionally)
- **Layer 5:** Application code copy (changes frequently)

Uvicorn startup time is approximately 2 seconds, which is within the `initialDelaySeconds` threshold of the Kubernetes liveness probe (10 seconds). The readiness probe has a shorter initial delay (5 seconds) and will correctly gate traffic until the asyncpg pool is initialised.